43

BÖLÜM: LLM PERFORMANSI, CACHING, MALIYET & GECIKME OPTIMIZASYONU | MODULE: LLM PERFORMANCE ENGINEERING: CACHING, MODEL ROUTING & COST OPTIMIZATION

Prompt Caching & Semantik Önbellekleme (Redis Semantic Cache)

Prompt Caching & Semantic Caching: Slashing LLM Incurred Costs & Latency

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restoranda her gelen müşteri 'Günün çorbası ne?' diye sorduğunda garsonun her seferinde mutfağa kadar 100 metre yürüyüp aşçıya sormasını istemezsiniz (Gereksiz LLM çağrısı)! Garson çorbanın adını önlüğünün cebindeki küçük kağıda yazar (Önbellek - Cache). Biri 'Günün çorbası ne?' veya 'Bugün hangi çorba var?' dediğinde (aynı anlama gelen farklı cümleler - Semantik Önbellek), garson mutfağa hiç gitmeden cebindeki kağıda bakıp 1 saniyede 'Mercimek!' der! Hem mutfak yorulmaz, hem müşteri beklemez, hem de patronun cebinden para çıkmaz!

🇬🇧 ENGLISH (ELI5)

If 5,000 customers walk into a cafe asking 'What is today's soup?', the waiter shouldn't sprint to the kitchen to ask the executive chef 5,000 times (expensive, redundant LLM API calls). The waiter scribbles the answer on a notepad in their pocket (Cache). When a customer asks 'What soup do you have?' or 'Tell me today's soup special' (semantically identical intent), the waiter reads the notepad instantly. Semantic Caching intercepts identical meaning queries in 5ms at $0 cost!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Önbellekleme Mimarisi Katmanları: 1) Tam Eşleşme (Exact Cache): Sorgunun SHA-256 hash'i Redis'te aranır. Çok kırılgandır (küçük bir noktalama işareti veya harf farkı cache'i bozar). 2) Semantik Önbellek (Redis Semantic Cache): Gelen sorgu embed edilir; Redis VSS (Vector Similarity Search) ile mevcut sorgu vektörleri arasında Cosine Similarity > 0.95 olan bir önceki yanıt varsa doğrudan önbellekten dönülür. Gecikme 1500ms'den 5ms'ye iner, maliyet %100 sıfırlanır! 3) OpenAI Prompt Caching: OpenAI 1024 token üzerindeki statik prefix'leri otomatik tanır; %50-80 indirim ve %80 daha hızlı TTFT sağlar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Multi-Tier Caching Architecture: 1) Exact Match Key-Value Cache: Hashes raw prompt text via SHA-256. Highly brittle: trivial whitespace discrepancies cause cache misses. 2) Semantic Caching (Redis VSS / GPTCache): Queries are embedded and compared against an index of historical queries. If Cosine Similarity exceeds a strict threshold (e.g. >=0.96), the precomputed response is served from memory. Latency drops from 2,000ms to <10ms, eliminating 100% of LLM billing for that request. 3) OpenAI Native Prefix Prompt Caching: Automatically caches common static prefix tokens >= 1,024 tokens across requests, slashing prompt token costs by 50% with zero engineering overhead.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25: Semantik Önbellek (Semantic Cache) Karar Motoru
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class SemanticCacheEngine {
    private final Map<String, String> exactCache = new ConcurrentHashMap<>();

    public record CacheLookupResult(boolean isHit, String response, long latencyMillis) {}

    public CacheLookupResult query(String userPrompt) {
        long start = System.currentTimeMillis();

        // Önce hızlı bellek içi / Redis kontrolü
        if (exactCache.containsKey(userPrompt.trim().toLowerCase())) {
            long duration = System.currentTimeMillis() - start;
            return new CacheLookupResult(true, exactCache.get(userPrompt.trim().toLowerCase()),
duration);
        }

        // Cache Miss: OpenAI'a gitmeli
        return new CacheLookupResult(false, null, 0);
    }

    public void put(String userPrompt, String llmResponse) {
        exactCache.put(userPrompt.trim().toLowerCase(), llmResponse);
    }
}

void main() {
    var cache = new SemanticCacheEngine();
    cache.put("şirket seyahat politikası", "Otel limiti günlük 150$'dır.");

    var result = cache.query("şirket seyahat politikası");
    IO.println("Cache Hit Durumu: " + (result.isHit() ? "✅ HIT (Sıfır Maliyet)" : "❌ MISS"));
    IO.println("Gecikme Süresi: " + result.latencyMillis() + " ms");
    IO.println("Cevap: " + result.response());
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Eşik Değeri Tehlikesi (Threshold Drift): Semantik önbellekte benzerlik eşiğini (Threshold) 0.90'ın altına çekerseniz, 'Ali'nin maaşı nedir?' sorusunun cevabı 'Veli'nin maaşı nedir?' sorusuna önbellekten yanlışlıkla servis edilebilir! Semantik önbellek eşiği daima en az 0.95 - 0.97 seviyesinde tutulmalıdır.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Semantic threshold danger: Lowering the semantic cache threshold below 0.95 causes dangerous cross-chatter (e.g. 'What is Alice's salary?' returning cached data for 'What is Bob's salary?'). Keep production similarity cutoffs strictly calibrated at >=0.96 to preserve semantic accuracy.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Semantik önbellek, sık sorulan kurumsal sorularda API maliyetini %40-%70 düşürür.
- Gecikme 2000ms'den 5ms'ye düşerek mükemmel bir kullanıcı deneyimi sunar.
- OpenAI Prompt Caching'den faydalanmak için sistem prompt'ları sabit tutulmalıdır.

🎯 Key Takeaways for the Interview

- Semantic caching cuts enterprise LLM operational costs by 40-70% on recurring traffic.
- Cached responses serve in sub-10ms, delivering instantaneous user responsiveness.
- Keep system prompts static and deterministic to exploit OpenAI's native prefix cache discounts.